

AI Chess Game

Created by Zain Sohail



Project overview

This project consists of the development of an AI-powered chess game using Python and Pygame, where the player competes against a computer controlled opponent. The aim is to create a fully functional chess game with a simple interface while exploring how artificial intelligence and algorithms can be used to make decisions in a competitive game.

Chess is a complex strategy game that requires players to consider multiple possible moves and predict how their opponent may respond. Developing a computer opponent therefore provides an opportunity to explore how algorithms can be used to solve problems and make decisions based on the current state of a game.

I developed this application as more than just a digital chess game. The goal was to create a chess game that combines engaging gameplay with an AI-controlled opponent. Through this project, I aim to explore how artificial intelligence and decision-making systems can be used to create an interactive and challenging chess experience

Minimax Algorithm

The Minimax algorithm is a decision making algorithm commonly used in games where two players compete against each other. It works by looking at possible moves for both players and predicting the possible outcomes of those moves. The algorithm assumes that both players will try to make the best move available to them. One player tries to maximise the score, while the other tries to minimise it. By comparing these possible outcomes, the algorithm can choose the move that is most likely to give the best result.

In my project, I implemented Minimax so that the AI controls the Black pieces and acts as the maximising player, while the human controls the White pieces and acts as the minimising player. The AI looks through a number of possible moves and considers how the player could respond to them. I also created a board evaluation system where each chess piece is given a value, with a pawn worth 1 point, a knight and bishop worth 3 points, a rook worth 5 points, a queen worth 9 points and a king worth 100 points. The algorithm uses these values to compare different board positions and calculate whether the position is more favourable to Black or White. The AI then uses these scores to select the move that gives it the best possible outcome.

Alpha-Beta Pruning

Alpha-Beta pruning is a technique that can be used alongside the Minimax algorithm. When playing chess, there are a very large number of possible moves that could be explored. Checking every possible position would take a long time, especially as the search goes deeper. Alpha-Beta pruning makes the Minimax algorithm more efficient by stopping it from exploring moves that are no longer relevant to the final decision.

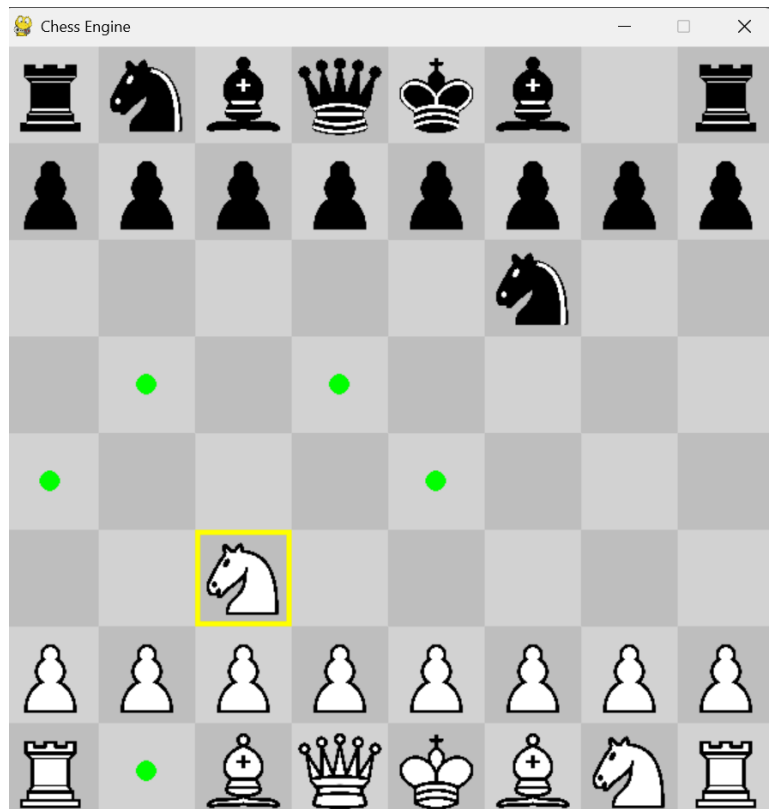
I implemented Alpha-Beta pruning into my Minimax algorithm using two values called Alpha and Beta. Alpha keeps track of the best score that the maximising player can currently guarantee, while Beta keeps track of the best score that the minimising player can guarantee. As the AI searches through possible moves, these values are updated. If the algorithm determines that a branch cannot produce a better result than a move that has already been considered, it stops exploring that branch. This reduces the number of positions that need to be checked, allowing the AI to search more efficiently while still selecting the same best move that Minimax would have chosen.

Key Features



Main game

Legal moves





```
def minimax(depth, alpha, beta, maximizing_player):
    if depth == 0:
        return evaluate_board()

    if maximizing_player:
        best_score = -10000
        moves = get_all_legal_moves("black")

        if len(moves) == 0:
            if is_in_check("black"):
                return -10000

            return 0

        for move in moves:
            end_row = move[2]
            end_column = move[3]

            captured_piece = board[end_row][end_column]
            original_piece = board[move[0]][move[1]]

            make_move(move)

            score = minimax(depth - 1, alpha, beta, False)

            undo_move(move, captured_piece, original_piece)

            if score > best_score:
                best_score = score
```

Minimax algorithm exploring possible moves

Alpha-Beta pruning reducing the number of unnecessary moves

```
        if best_score < beta:
            beta = best_score

        if beta <= alpha:
            break

    return best_score
```

```
        if best_score > alpha:
            alpha = best_score

        if beta <= alpha:
            break

    return best_score
```

Use of AI and External Resources

As this was my first time using Pygame properly in a large project such as this, I used GeeksforGeeks to help me understand certain Pygame concepts and how to implement different features within my project. I also used AI to help me understand the Minimax algorithm and Alpha-Beta pruning, as well as to better understand certain programming concepts that I found difficult.

I used these resources as a way to support my learning and development throughout the project. I then applied what I learned to my own project and tested the features to make sure they worked correctly.